K.R. Mangalam University

School of Engineering & Technology

ENSP465 — New Age Programming Languages Lab

Capstone Project — Phase II: Implementation & Intermediate Demonstration

TaskFlow

A Distributed Job Scheduler in Go

Team Members

S.No.	Student Name	Roll Number
1	Adeela Azeez	2301010282
2	Archi Garg	2301010346
3	Kirti Sahni	2301010236
4	Abhishek Raj	2301010111
5	Vinay Pratap Singh	2301010231

Semester: VII | Academic Year: 2026–27

Submission Date: 10 November 2026

Faculty: Dr. Hemraj Kumawat

GitHub Repository: https://github.com/CRICSTAR1718/TaskFlow

1. Project Overview

TaskFlow is a distributed job scheduling system written in Go. It lets users submit background jobs (immediate, delayed, or recurring via cron expressions), runs them concurrently across a cluster of worker nodes, and tracks every job in real time through a React dashboard. Jobs are prioritised, bounded by timeouts, retried automatically with exponential backoff on failure, and recovered if the worker executing them crashes.

The problem it addresses is that cron and ad-hoc scripts provide no distribution, retries, priorities, timeouts or visibility. TaskFlow fills this gap with a lightweight system that needs only Go binaries, Redis and PostgreSQL.

GitHub Repository: https://github.com/CRICSTAR1718/TaskFlow

2. Objectives

- Execute at least 100 jobs in parallel across 3 or more worker nodes using a goroutine pool.
- Support three priority levels, with high-priority jobs always dequeued first.
- Retry failed jobs automatically with exponential backoff and move exhausted jobs to a dead-letter state.
- Enforce per-job timeouts and recover jobs from a crashed worker within 15 seconds.
- Stream job status changes to a dashboard in under one second using WebSockets.

3. Key Features

Feature	Description
Job submission	REST API and dashboard form; immediate, delayed and cron-recurring jobs with JSON payloads
Priority queues	Three Redis sorted-set queues (high, normal, low); atomic claim using a Lua script so no job is taken twice
Distributed workers	Any number of worker nodes, each running a configurable goroutine pool
Timeouts	Every job runs under context.WithTimeout and is cancelled when the deadline passes
Automatic retries	Exponential backoff with jitter, a configurable retry limit and a dead-letter state
Failure recovery	Worker heartbeats with a TTL; jobs of a dead worker are detected and re-queued
Real-time tracking	Job events published over Redis pub/sub and pushed to the dashboard by a WebSocket hub
Dashboard	Overview statistics, job list with filters and history, worker health, and a job submission form

4. Technologies Used

Layer	Technology
Language	Go (all backend services)
HTTP / routing	go-chi/chi, gorilla/websocket
Queue and coordination	Redis (go-redis v9): sorted sets, Lua scripts, TTL keys, pub/sub, locks
Database	PostgreSQL (pgx v5), golang-migrate for schema migrations
Scheduling	robfig/cron v3 for recurring jobs
Frontend	React, TypeScript, Vite, TailwindCSS
DevOps	Docker, Docker Compose, GitHub Actions
Testing	Go testing package, race detector, load-test script

5. System Architecture

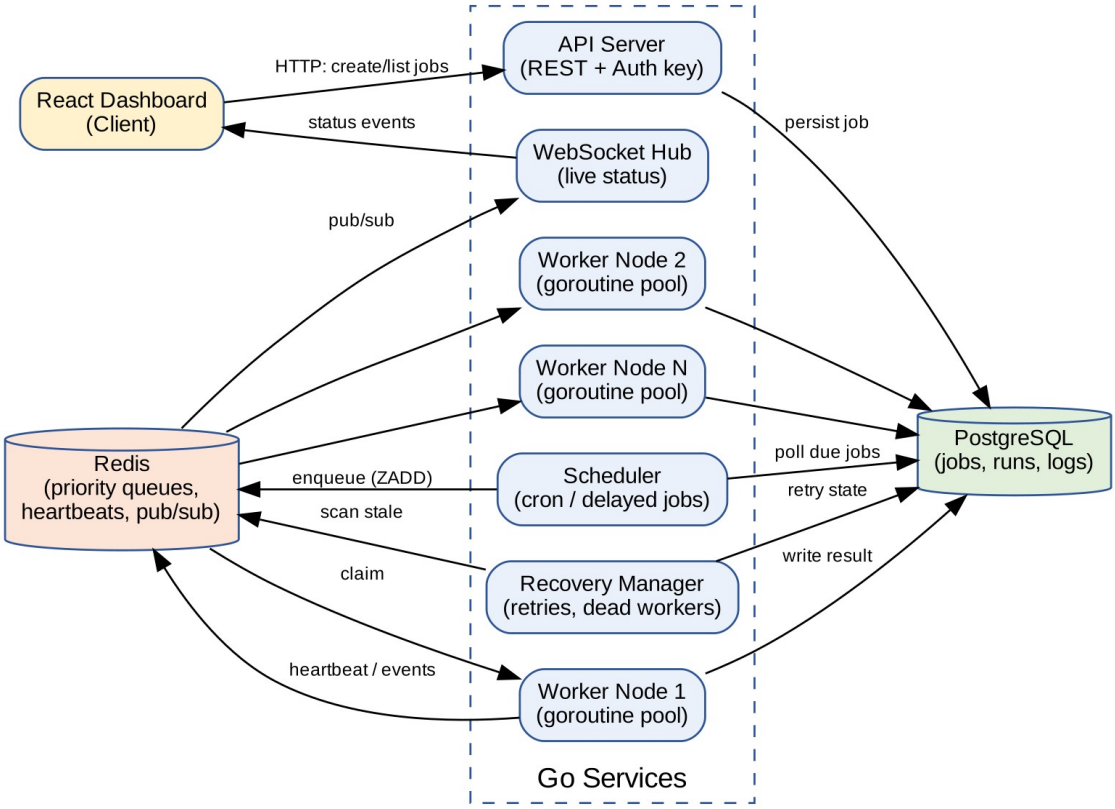


Figure 1: TaskFlow High-Level Architecture

Figure 1: TaskFlow architecture

The system is split into four Go services plus the React client. The API server validates and stores jobs in PostgreSQL. The scheduler polls PostgreSQL for due jobs and pushes them into Redis priority queues. Worker nodes claim jobs from Redis and execute them. The recovery manager handles retries and dead workers. The WebSocket hub relays job events to the dashboard.

6. Implementation

6.1 Module Summary

Module	Implementation	Owner
API server (cmd/api)	chi router with API-key middleware, request logging and CORS; endpoints to create, list, inspect, cancel and retry jobs, plus stats, workers and health	Kirti Sahni
Store (internal/store)	pgx connection pool; migrations for jobs, job_runs and workers; FOR UPDATE SKIP LOCKED to fetch due jobs safely	Kirti Sahni
Queue (internal/queue)	Redis sorted-set queues; Lua script that atomically pops the highest-priority job and records it as in-flight	Archi Garg
Scheduler (cmd/scheduler)	One-second ticker; leader lock so only one instance enqueues; cron next-run calculation	Archi Garg
Worker (cmd/worker)	Fetcher goroutine, buffered job channel, executor goroutine pool, timeouts, heartbeat, graceful shutdown	Adeela Azeez
Handlers (internal/handlers)	JobHandler interface and registry; sample handlers (sleep, http_ping, fail, email_mock, report)	Adeela Azeez
Recovery (cmd/recovery)	Backoff calculation, dead-letter handling, heartbeat scan and re-queue of orphaned jobs	Abhishek Raj
WebSocket hub (internal/ws)	Redis pub/sub subscriber, per-client writer goroutines, periodic stats broadcast	Vinay Pratap Singh
Dashboard (web/)	React pages: Overview, Jobs, Workers, Submit Job; live updates via WebSocket	Vinay Pratap Singh

6.2 Job Lifecycle

A job moves through the states PENDING → QUEUED → RUNNING → SUCCEEDED. A failed attempt moves it to RETRYING, after which the recovery manager schedules the next attempt with exponential backoff (base × 2^attempt, with jitter). When the retry limit is exhausted, or the handler reports a permanent error, the job becomes DEAD. Pending jobs can also be CANCELLED by the user.

6.3 Go Concurrency Features Used

- **Goroutines:** executor pool, queue fetcher, heartbeat sender, scheduler ticker and WebSocket client writers.
- **Buffered channels and select:** hand-off of claimed jobs to executors with back-pressure; a single select waits on new jobs, shutdown and the heartbeat ticker.
- **context:** per-job timeouts, cancellation of running jobs and graceful shutdown.

- **sync.WaitGroup and Mutex:** draining in-flight jobs on shutdown and protecting the WebSocket client registry.
- **Interfaces:** JobHandler for pluggable job types; narrow Queue and Store interfaces so workers can be tested with fakes.

7. Conclusion and Next Steps

Phase II delivers the working core of TaskFlow: job submission, priority queuing, concurrent execution on multiple workers, and a live dashboard. In Phase III we will complete remaining reliability work, add integration and load tests with documented benchmarks, containerise the whole system with Docker Compose, deploy it to a cloud VM, and finalise documentation and viva preparation.

AI acknowledgment: Parts of the design, code and documentation of this project were developed with the assistance of AI tools. All code was reviewed, tested and understood by the team members responsible for each module.